COMPARATIVE ANALYSIS OF SMOTE ON LOGISTIC REGRESSION AND RANDOM FOREST FOR FRAUD DETECTION ON FINANCIAL DATA

1. DATASET
Financial Transactions

2. PREPROCESSING
- Train-Test Split
- Feature Scaling

Highly Imbalanced Dataset

4. SMOTE
Synthetic Minority Oversampling Technique

5. MODEL TRAINING
- Logistic Regression
- Random Forest

6. EVALUATION
- Accuracy
- Precision
- Recall
- F1 Score

FIND THE BEST MODEL FOR FRAUD DETECTION

AUTHOR
ROUNAK

MAY 2026

Contents

1 Introduction **4**

2 Dataset Description and Exploratory Data Analysis **5**

3 Data Preprocessing and Experimental Methodology **6**

3.1 Train-Test Split . . . 6

3.2 Feature Scaling . . . 6

3.3 Handling Class Imbalance Using SMOTE . . . 7

3.4 Experimental Scenarios . . . 7

3.5 Evaluation Metrics . . . 7

3.5.1 Confusion Matrix . . . 7

3.5.2 Accuracy . . . 8

3.5.3 Precision . . . 8

3.5.4 Recall . . . 8

3.5.5 F1 Score . . . 8

3.6 Importance of Metric Tradeoffs . . . 8

4 Machine Learning Algorithms and SMOTE **9**

4.1 Logistic Regression . . . 9

4.1.1 Mathematical Working . . . 9

4.1.2 Implementation . . . 9

4.2 Random Forest . . . 9

4.2.1 Decision Tree Foundation . . . 9

4.2.2 How Random Forest Works . . . 10

4.2.3 Implementation . . . 10

4.3 Synthetic Minority Oversampling Technique . . . 10

4.3.1 Mathematical Working of SMOTE . . . 10

4.3.2 Backend Workflow . . . 11

4.3.3 Implementation . . . 11

4.4 Why SMOTE Affected the Models Differently . . . 11

5 Experimental Results and Comparative Analysis **12**

5.1 Logistic Regression without SMOTE . . . 12

5.2 Logistic Regression with SMOTE . . . 12

5.3 Random Forest without SMOTE . . . 13

5.4 Random Forest with SMOTE . . . 13

5.5 Overall Model Comparison . . . 14

5.6 False Positive and False Negative Analysis . . . 14

5.7 Precision-Recall Tradeoff . . . 15

5.8 Why SMOTE Helped Logistic Regression . . . 15

5.9 Why SMOTE Did Not Improve Random Forest Significantly . . . 15

5.10 Key Findings . . . 15

6 Conclusion **17**

Abstract

Credit card fraud detection is one of the most critical applications of Machine Learning in financial systems due to the continuously increasing volume of digital transactions and evolving fraudulent activities. A major challenge in fraud detection datasets is severe class imbalance, where fraudulent transactions represent only a very small fraction of total transactions. Traditional Machine Learning models often become biased toward the majority class, resulting in poor fraud detection capability despite high overall accuracy. This project presents a comparative

analysis of the impact of Synthetic Minority Oversampling Technique (SMOTE) on two widely used classification algorithms: Logistic Regression and Random Forest. The experiments were conducted on a highly imbalanced credit card transaction dataset containing 284,807 transactions, of which only 492 were fraudulent. Four experimental scenarios were evaluated: Logistic Regression without SMOTE, Logistic Regression with SMOTE, Random Forest without SMOTE, and Random Forest with SMOTE. The study investigates the effect of oversampling on model

behavior using evaluation metrics including Accuracy, Precision, Recall, F1 Score, False Positives, and False Negatives. Experimental results show that SMOTE significantly improved the Recall of Logistic Regression from 64.29% to 91.84%, reducing missed fraud cases substantially. However, this improvement came at the cost of a dramatic increase in false positives. In contrast, Random Forest demonstrated strong fraud detection capability even without oversampling and achieved the best overall balance between Precision, Recall, and F1 Score. The

findings demonstrate that imbalance handling techniques such as SMOTE are highly model-dependent and do not universally improve performance across all algorithms. The project highlights the critical tradeoff between fraud detection sensitivity and false alarm generation in real-world financial fraud detection systems.

Keywords: Fraud Detection, Machine Learning, SMOTE, Logistic Regression, Random Forest, Imbalanced Learning, Classification, Financial Data Analytics

Project Repository: GitHub Repository Link

1 Introduction

The rapid growth of digital financial systems and online transaction platforms has significantly increased the volume of electronic payments worldwide. While digital banking and card-based payment systems provide convenience and speed, they have also created new opportunities for fraudulent activities. Credit card fraud has become a major financial threat for banks, financial institutions, merchants, and customers due to the large economic losses associated with unauthorized transactions.

Traditional fraud detection systems primarily relied on rule-based mechanisms and manually engineered verification strategies. However, modern fraud patterns continuously evolve in complexity, making static rule-based systems insufficient for large-scale real-time fraud detection. As a result, Machine Learning-based fraud detection systems have become increasingly important because of their ability to automatically learn hidden patterns and anomalies from historical transaction data.

Fraud detection is fundamentally a binary classification problem in which each transaction is classified as either fraudulent or non-fraudulent. One of the most significant challenges in fraud detection datasets is severe class imbalance. In real-world financial datasets, fraudulent transactions typically represent only a very small fraction of the total transaction volume. This imbalance causes Machine Learning algorithms to become biased toward the majority class, often resulting in deceptively high accuracy while failing to correctly identify fraudulent transactions.

In highly imbalanced datasets, traditional evaluation metrics such as Accuracy alone are insufficient for measuring model performance. For fraud detection systems, metrics such as Precision, Recall, F1 Score, False Positives, and False Negatives become critically important. In particular, Recall plays a major role because false negatives represent fraudulent transactions that remain undetected by the system, potentially causing significant financial losses.

To address the imbalance problem, various imbalance handling techniques have been developed. One of the most widely used methods is Synthetic Minority Oversampling Technique (SMOTE), which generates synthetic samples for the minority class to balance the dataset during training. Although SMOTE is commonly used to improve minority class learning, its effectiveness may vary depending on the Machine Learning algorithm being used.

This project presents a comparative analysis of the impact of SMOTE on two popular Machine Learning algorithms:

- Logistic Regression
- Random Forest

The study evaluates model performance under four experimental scenarios:

1. Logistic Regression without SMOTE
2. Logistic Regression with SMOTE
3. Random Forest without SMOTE
4. Random Forest with SMOTE

The objective of this project is not only to improve fraud detection performance but also to understand how imbalance handling techniques affect different Machine Learning models. The study particularly focuses on the tradeoff between fraud detection sensitivity and false alarm generation.

The experimental results demonstrate that SMOTE significantly improved the Recall of Logistic Regression but also introduced a large number of false positives. In contrast, Random Forest achieved strong performance even without oversampling and showed only limited improvement after applying SMOTE. These findings indicate that imbalance handling techniques are highly model-dependent and that appropriate model selection can sometimes outperform oversampling strategies.

This project highlights the practical challenges involved in real-world fraud detection systems and provides insights into balancing fraud detection effectiveness with operational reliability.

2 Dataset Description and Exploratory Data Analysis

The dataset used in this project contains anonymized credit card transactions made by European cardholders over a two-day period. The dataset is highly imbalanced, with fraudulent transactions representing only a very small fraction of the total observations.

The dataset consists of 284,807 transactions with 31 columns, including 30 input features and one target variable named `Class`. The target variable represents whether a transaction is fraudulent or non-fraudulent.

Property	Value
Total Transactions	284,807
Fraud Transactions	492
Non-Fraud Transactions	284,315
Fraud Percentage	0.172%
Number of Features	30
Target Variable	Class

Table 1: Dataset Statistics

The dataset contains the following feature categories:

- **Time**: Time elapsed between transactions
- **Amount**: Transaction amount
- **V1 − V28**: PCA-transformed anonymized features
- **Class**: Target label

The features `V1` to `V28` were transformed using Principal Component Analysis (PCA) to protect sensitive financial information. Since these features are already standardized through PCA transformation, additional scaling was only applied to the `Time` and `Amount` columns during preprocessing.

One of the most critical observations from the dataset is the severe class imbalance problem. Fraudulent transactions account for only 492 out of 284,807 total transactions.

The class distribution is shown in Figure 1.

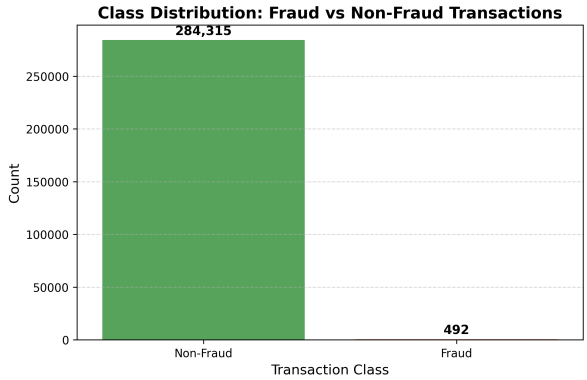

Figure 1: Class Distribution of Fraud and Non-Fraud Transactions

The distribution clearly shows that the dataset is dominated by non-fraudulent transactions. Such imbalance creates a strong bias toward the majority class during model training. As a result, Machine Learning algorithms may achieve very high accuracy while still failing to detect fraudulent transactions effectively.

For example, a naive model that predicts every transaction as non-fraudulent would still achieve an accuracy greater than 99%, despite completely failing to detect fraud cases. This demonstrates why Accuracy alone is not an appropriate evaluation metric for fraud detection systems.

The imbalance problem motivates the use of specialized imbalance handling techniques such as SMOTE, which is explored later in this project.

During exploratory data analysis, the dataset was also checked for missing values and data consistency. No missing values were found in any feature column.

The dataset information analysis confirmed that:

- All features were numerical
- No null values existed
- The dataset was already clean and structured
- Fraud class imbalance was extremely severe

These observations formed the foundation for the preprocessing and modeling stages performed later in the project.

3 Data Preprocessing and Experimental Methodology

Before training the Machine Learning models, several preprocessing and experimental preparation steps were performed to ensure proper model evaluation, fair comparison between algorithms, and reliable fraud detection analysis. Since the dataset used in this project was highly imbalanced, careful preprocessing and validation strategies were necessary to prevent biased learning and misleading evaluation results.

The preprocessing workflow involved:

- Train-test splitting
- Feature scaling
- Class imbalance handling using SMOTE
- Model training
- Performance evaluation

Each preprocessing stage plays a critical role in the overall Machine Learning pipeline.

3.1 Train-Test Split

The dataset was divided into training and testing subsets using an 80:20 ratio.

- 80% of the data was used for model training
- 20% of the data was used for model testing

The purpose of splitting the dataset is to evaluate how well the trained model generalizes to unseen transactions. The model learns patterns from the training data and is then evaluated on completely unseen testing data.

The following implementation was used:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

The parameter `stratify=y` is extremely important for imbalanced datasets. It ensures that both the training and testing datasets preserve the same fraud-to-non-fraud ratio as the original dataset.

Without stratification, one subset might accidentally contain very few fraud samples, causing unreliable evaluation results.

The parameter `random_state=42` ensures reproducibility. This means the same train-test split can be recreated every time the experiment is executed.

3.2 Feature Scaling

Feature scaling was applied only to the `Time` and `Amount` columns.

The PCA-transformed features (`V1 -- V28`) were already standardized during dataset preparation. Therefore, additional scaling was unnecessary for those features.

The scaling implementation used was:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train[["Time", "Amount"]] = scaler.fit_transform(
    X_train[["Time", "Amount"]]
)

X_test[["Time", "Amount"]] = scaler.transform(
    X_test[["Time", "Amount"]]
)
```

Standardization transforms features using the following equation:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x = original feature value
- μ = mean of the feature
- σ = standard deviation
- z = standardized feature value

The goal of standardization is to transform features so that they have:

- Mean approximately equal to 0
- Standard deviation approximately equal to 1

Feature scaling is important because Machine Learning algorithms may become biased toward features with larger numerical ranges.

For example:

- Transaction Amount values may range into thousands
- PCA features may range only between small decimal values

Without scaling, the larger-valued features could dominate the learning process.

Scaling is especially important for Logistic Regression because it is distance-sensitive and optimization-based. Random Forest is less sensitive to feature scaling because it uses tree-based splitting rather than distance calculations.

3.3 Handling Class Imbalance Using SMOTE

One of the biggest challenges in this project was the severe class imbalance problem.

The dataset contained:

- 284,315 non-fraud transactions
- Only 492 fraud transactions

This means fraudulent transactions represented only approximately 0.17% of the total dataset.

Such imbalance causes Machine Learning models to become heavily biased toward the majority class. A model may achieve extremely high Accuracy simply by predicting every transaction as non-fraud.

To address this issue, Synthetic Minority Oversampling Technique (SMOTE) was used.

The implementation used was:

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train,
    y_train
)
```

Unlike simple oversampling, SMOTE does not duplicate fraud samples. Instead, it creates synthetic minority samples between existing fraud samples using nearest neighbor interpolation.

The general SMOTE equation is:

$$x_{new} = x_i + \lambda(x_{neighbor} - x_i)$$

Where:

- x_i = existing minority sample
- $x_{neighbor}$ = nearest minority neighbor
- λ = random value between 0 and 1
- x_{new} = generated synthetic sample

This process helps create more balanced training data while preserving the general structure of the fraud class.

SMOTE was applied only to the training data.

This is a critical step because applying SMOTE before train-test splitting can cause data leakage. In such cases, synthetic samples generated using training information may indirectly influence the testing data, producing unrealistically high evaluation scores.

The correct workflow followed in this project was:

$$\text{Dataset} \rightarrow \text{Train-Test Split} \rightarrow \text{Scaling}$$

$$\downarrow$$

$$\text{SMOTE on Training Data} \rightarrow \text{Model Training} \rightarrow \text{Evaluation}$$

3.4 Experimental Scenarios

Four experimental scenarios were evaluated:

1. Logistic Regression without SMOTE
2. Logistic Regression with SMOTE
3. Random Forest without SMOTE
4. Random Forest with SMOTE

This experimental setup allowed direct comparison between:

- Linear vs nonlinear models
- Balanced vs imbalanced training
- Recall-focused vs Precision-focused behavior

The primary objective was to analyze how SMOTE influences different Machine Learning algorithms and whether oversampling universally improves fraud detection performance.

3.5 Evaluation Metrics

Model performance was evaluated using multiple classification metrics rather than relying only on Accuracy.

This is extremely important because fraud detection datasets are highly imbalanced.

3.5.1 Confusion Matrix

The Confusion Matrix is one of the most important evaluation tools in classification problems.

It compares predicted classes against actual classes.

	Predicted Non-Fraud	Predicted Fraud
Actual Non-Fraud	True Negative (TN)	False Positive (FP)
Actual Fraud	False Negative (FN)	True Positive (TP)

Table 2: Confusion Matrix Structure

Each component has a practical meaning:

- **True Positive (TP)**: Fraud correctly detected as fraud.
- **True Negative (TN)**: Non-fraud correctly detected as non-fraud.
- **False Positive (FP)**: Legitimate transaction incorrectly flagged as fraud.
- **False Negative (FN)**: Fraud transaction incorrectly classified as legitimate.

In fraud detection systems:

- False Negatives are dangerous because fraud goes undetected.
- False Positives are also costly because genuine customers may get blocked or interrupted.

Therefore, fraud detection systems must carefully balance fraud sensitivity with operational reliability.

3.5.2 Accuracy

Accuracy measures the proportion of correctly classified transactions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Although Accuracy is commonly used, it becomes misleading in highly imbalanced datasets.

For example, predicting every transaction as non-fraud would still produce Accuracy above 99% in this dataset, despite completely failing to detect fraud.

Therefore, Accuracy alone is not sufficient for fraud detection evaluation.

3.5.3 Precision

Precision measures how many predicted fraud transactions were actually fraudulent.

$$Precision = \frac{TP}{TP + FP}$$

High Precision means:

- Few false fraud alarms
- More trustworthy fraud predictions

Low Precision means the model is aggressively predicting fraud, causing many legitimate transactions to be incorrectly blocked.

3.5.4 Recall

Recall measures how many actual fraud transactions were successfully detected.

$$Recall = \frac{TP}{TP + FN}$$

Recall is one of the most important metrics in fraud detection.

High Recall means:

- Fewer fraud transactions escape detection
- Better fraud-catching capability

Low Recall means the model is missing fraud cases.

In financial systems, low Recall can directly translate into financial losses.

3.5.5 F1 Score

F1 Score combines Precision and Recall into a single metric.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1 Score becomes useful when balancing:

- Fraud detection sensitivity
- False alarm reduction

A high F1 Score indicates that both Precision and Recall are reasonably strong.

3.6 Importance of Metric Tradeoffs

In fraud detection systems, improving one metric often negatively affects another.

For example:

- Increasing Recall may increase False Positives
- Increasing Precision may reduce fraud detection sensitivity

This tradeoff was clearly observed in the experimental results of this project.

SMOTE significantly improved Recall for Logistic Regression but also caused a large increase in False Positives. Random Forest, on the other hand, maintained a stronger balance between Precision and Recall even without aggressive oversampling.

Therefore, the final model selection in fraud detection systems should depend not only on statistical performance but also on business requirements, operational costs, and acceptable fraud risk levels.

4 Machine Learning Algorithms and SMOTE

This section explains the core Machine Learning algorithms used in this project: Logistic Regression, Random Forest, and SMOTE. Logistic Regression and Random Forest were used as classification models, while SMOTE was used as an imbalance handling technique to generate synthetic fraud samples for the minority class.

4.1 Logistic Regression

Logistic Regression is a supervised Machine Learning algorithm commonly used for binary classification problems. In this project, the model classifies each transaction into one of two classes:

- Class 0: Non-Fraud Transaction
- Class 1: Fraud Transaction

Although the term "regression" is present in its name, Logistic Regression is used for classification. It estimates the probability that a transaction belongs to the fraud class.

4.1.1 Mathematical Working

Logistic Regression first calculates a linear combination of input features:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where $x_1, x_2, \ldots, x_n$ represent the input features such as `Time`, `Amount`, and PCA-transformed features `V1` to `V28`. The terms $\beta_1, \beta_2, \ldots, \beta_n$ are the learned coefficients, and β_0 is the intercept.

The linear score z is then passed through the sigmoid function:

$$p = \frac{1}{1 + e^{-z}}$$

The sigmoid function converts the linear output into a probability between 0 and 1.

- If $p \geq 0.5$, the transaction is classified as fraud.
- If $p < 0.5$, the transaction is classified as non-fraud.

The sigmoid curve is shown in Figure 2.

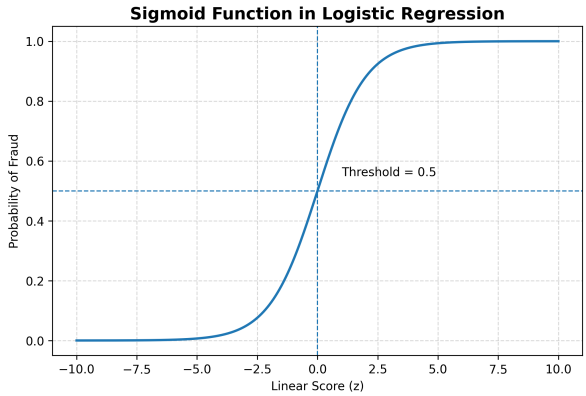

Figure 2: Sigmoid Function Used in Logistic Regression

In simple terms, Logistic Regression tries to learn a decision boundary that separates fraud and non-fraud transactions. Since it is a linear model, it works well when the classes can be separated using a relatively simple boundary. However, in highly imbalanced datasets, the model may become biased toward the majority class because it sees far more non-fraud examples than fraud examples.

4.1.2 Implementation

The Logistic Regression model was implemented using Scikit-learn:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

The parameter `max_iter=1000` ensures that the optimization algorithm gets enough iterations to converge. The model learns coefficients for all input features during training and then uses those learned coefficients to classify unseen transactions.

4.2 Random Forest

Random Forest is an ensemble learning algorithm based on multiple Decision Trees. Instead of relying on a single tree, Random Forest builds many trees and combines their predictions to make a final decision.

In this project, Random Forest was used as a stronger nonlinear classifier to compare against Logistic Regression.

4.2.1 Decision Tree Foundation

A Decision Tree splits the data based on feature conditions. For example, a tree may learn rules such as:

$$\text{If } V14 < threshold, \text{ go left; otherwise, go right.}$$

At each split, the tree tries to separate fraud and non-fraud transactions as effectively as possible. One common criterion used for splitting is Gini Impurity.

$$Gini = 1 - \sum_{i=1}^{C} p_i^2$$

where p_i is the proportion of class i in a node, and C is the number of classes.

A low Gini value means the node is more pure, meaning it mostly contains one class. A high Gini value means the node contains a mixture of classes.

4.2.2 How Random Forest Works

Random Forest improves Decision Trees using two main techniques:

- **Bootstrap Sampling**: Each tree is trained on a random sample of the training data.
- **Random Feature Selection**: At each split, only a random subset of features is considered.

This creates multiple diverse trees. Each tree makes its own prediction, and the final output is decided using majority voting.

$$\hat{y} = majority_vote(T_1, T_2, \ldots, T_m)$$

where $T_1, T_2, \ldots, T_m$ are individual decision trees.

The conceptual structure of Random Forest is shown in Figure 3.

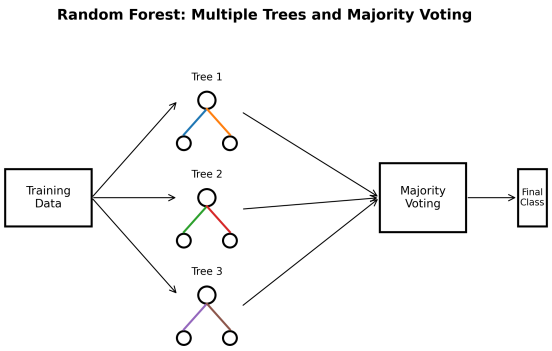

Figure 3: Random Forest Ensemble and Majority Voting

Random Forest is powerful because it can capture non-linear relationships and complex feature interactions. Unlike Logistic Regression, it is not restricted to a linear decision boundary. This is one reason Random Forest performed strongly in this project even without applying SMOTE.

4.2.3 Implementation

The Random Forest model was implemented as follows:

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
```

The parameter `n_estimators=100` means that 100 decision trees were built. The parameter `random_state=42` ensures reproducible results, and `n_jobs=-1` allows the model to use all available CPU cores for faster training.

4.3 Synthetic Minority Oversampling Technique

Synthetic Minority Oversampling Technique, commonly known as SMOTE, is an imbalance handling method used to increase the number of minority class samples.

In this project, fraud transactions are the minority class. Before SMOTE, the training data contained only 394 fraud samples compared to 227,451 non-fraud samples. This imbalance can cause models to learn the non-fraud class more strongly and ignore rare fraud patterns.

4.3.1 Mathematical Working of SMOTE

SMOTE does not simply duplicate fraud records. Instead, it creates synthetic fraud samples between existing fraud samples.

The general equation is:

$$x_{new} = x_i + \lambda(x_{neighbor} - x_i)$$

where:

- x_i is an existing minority class sample.
- $x_{neighbor}$ is one of its nearest minority class neighbors.
- λ is a random value between 0 and 1.
- x_{new} is the newly generated synthetic sample.

For example, if one fraud sample has a feature value of 10 and its nearest fraud neighbor has a value of 16, and if $\lambda = 0.5$, then:

$$x_{new} = 10 + 0.5(16 - 10) = 13$$

This new synthetic point lies between two real fraud samples. In the actual dataset, this process happens across all feature columns, creating complete synthetic transaction records.

The SMOTE sample generation concept is shown in Figure 4.

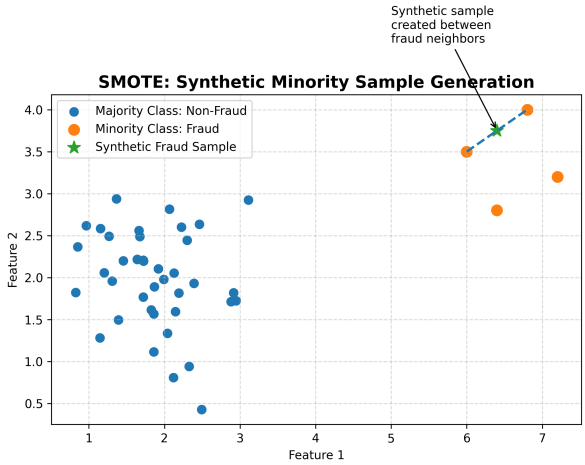

Figure 4: SMOTE Synthetic Sample Generation

4.3.2 Backend Workflow

Internally, SMOTE performs the following steps:

1. Identify the minority class.
2. Select a minority class sample.
3. Find nearest minority class neighbors.
4. Randomly select one neighbor.
5. Generate a synthetic sample between the selected sample and its neighbor.
6. Repeat until the minority class reaches the required size.

In this project, SMOTE transformed the training class distribution from:

$$\text{Non-Fraud: } 227451, \text{ Fraud: } 394$$

to:

$$\text{Non-Fraud: } 227451, \text{ Fraud: } 227451$$

This balanced dataset was then used to train the models.

4.3.3 Implementation

The SMOTE implementation used in the project was:

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train,
    y_train
)
```

The function `fit_resample()` first learns the minority class structure and then generates synthetic samples. SMOTE was applied only to the training data to avoid data leakage. The test data was kept unchanged so that evaluation was performed on real unseen transactions.

4.4 Why SMOTE Affected the Models Differently

SMOTE helped Logistic Regression significantly because Logistic Regression is a linear model and is more sensitive to class distribution. Before SMOTE, Logistic Regression saw very few fraud samples, so its decision boundary was influenced heavily by the majority non-fraud class. After SMOTE, the model received many more fraud-like samples, which shifted the decision boundary and improved Recall.

However, this also made Logistic Regression overly sensitive to fraud, causing a large increase in false positives.

Random Forest behaved differently. Since Random Forest builds many nonlinear decision trees, it can already capture complex and localized fraud patterns without needing a perfectly balanced dataset. Therefore, applying SMOTE to Random Forest gave only a small Recall improvement but reduced Precision by increasing false positives.

This demonstrates an important Machine Learning insight:

SMOTE is model-dependent and does not always improve every algorithm.

5 Experimental Results and Comparative Analysis

This section presents the experimental results obtained from four model configurations. The purpose of this comparison is to evaluate how SMOTE affects different Machine Learning algorithms in a highly imbalanced fraud detection problem.

The four evaluated scenarios are:

1. Logistic Regression without SMOTE
2. Logistic Regression with SMOTE
3. Random Forest without SMOTE
4. Random Forest with SMOTE

The models were evaluated using Accuracy, Precision, Recall, F1 Score, False Positives, and False Negatives. Since the dataset is highly imbalanced, the analysis focuses more on Precision, Recall, F1 Score, and confusion matrix behavior rather than Accuracy alone.

5.1 Logistic Regression without SMOTE

The first experiment used Logistic Regression directly on the imbalanced dataset without applying SMOTE.

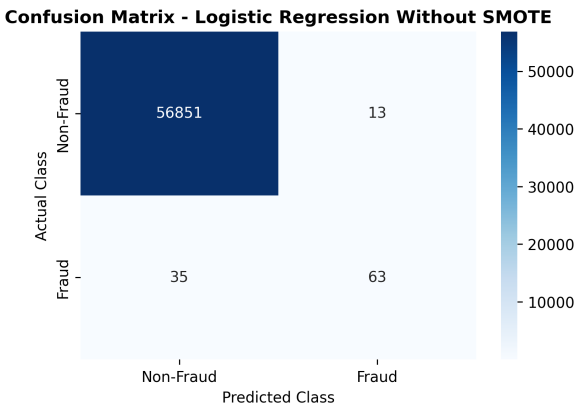

Figure 5: Confusion Matrix for Logistic Regression without SMOTE

Metric	Value
Accuracy	99.91%
Precision	82.89%
Recall	64.29%
F1 Score	72.41%
False Positives	13
False Negatives	35

Table 3: Logistic Regression without SMOTE Results

The model achieved very high Accuracy because most transactions in the dataset were non-fraudulent. However, the Recall value was only 64.29%, meaning the model missed a significant number of actual fraud cases.

From the confusion matrix, the model correctly detected 63 fraud transactions but failed to detect 35 fraud transactions. These 35 false negatives are critical because they represent fraudulent transactions that would pass through the system undetected.

This result shows that Logistic Regression without SMOTE was conservative in predicting fraud. It produced only 13 false positives, which means it generated few false alarms. However, the cost of this conservative behavior was a lower fraud detection rate.

5.2 Logistic Regression with SMOTE

In the second experiment, SMOTE was applied to the training data before training the Logistic Regression model.

Before SMOTE, the training class distribution was:

$$\text{Non-Fraud: } 227451, \text{ Fraud: } 394$$

After SMOTE, the training class distribution became:

$$\text{Non-Fraud: } 227451, \text{ Fraud: } 227451$$

This balanced training set allowed the Logistic Regression model to learn fraud patterns more aggressively.

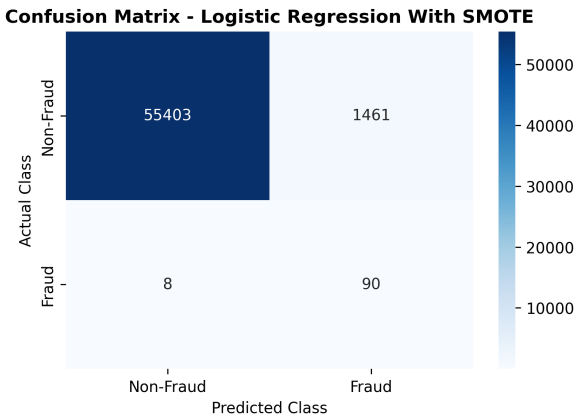

Figure 6: Confusion Matrix for Logistic Regression with SMOTE

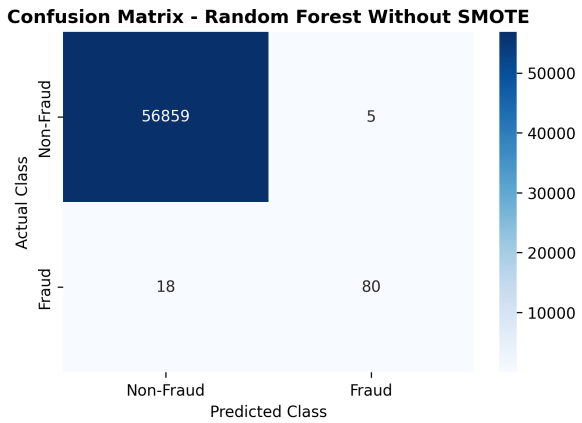

Figure 7: Confusion Matrix for Random Forest without SMOTE

Metric	Value
Accuracy	97.42%
Precision	5.80%
Recall	91.84%
F1 Score	10.92%
False Positives	1461
False Negatives	8

Table 4: Logistic Regression with SMOTE Results

Metric	Value
Accuracy	99.96%
Precision	94.12%
Recall	81.63%
F1 Score	87.43%
False Positives	5
False Negatives	18

Table 5: Random Forest without SMOTE Results

After applying SMOTE, Recall increased significantly from 64.29% to 91.84%. This means the model became much better at detecting actual fraud transactions. The number of false negatives decreased from 35 to 8.

However, this improvement came with a major tradeoff. Precision dropped from 82.89% to 5.80%. This happened because the model started predicting many more transactions as fraud, which increased the number of false positives from 13 to 1461.

This result shows that SMOTE helped Logistic Regression learn the minority fraud class better, but it also made the model overly sensitive to fraud. In a real financial system, this may reduce missed frauds but would create a large number of false alerts for genuine customers.

5.3 Random Forest without SMOTE

The third experiment used Random Forest without applying SMOTE.

Random Forest without SMOTE produced the strongest balanced result among the evaluated models. It achieved the highest Precision and F1 Score while maintaining strong Recall.

The model detected 80 fraud transactions and missed 18 fraud transactions. It also produced only 5 false positives, which is the lowest false positive count among all experiments.

Compared to Logistic Regression without SMOTE, Random Forest improved Recall from 64.29% to 81.63%, while also improving Precision from 82.89% to 94.12%. This demonstrates that Random Forest captured fraud patterns more effectively even without synthetic oversampling.

This stronger performance can be explained by the nonlinear and ensemble-based nature of Random Forest. Unlike Logistic Regression, Random Forest can learn complex decision boundaries and feature interactions, making it more suitable for structured fraud detection data.

5.4 Random Forest with SMOTE

The fourth experiment applied SMOTE before training the Random Forest model.

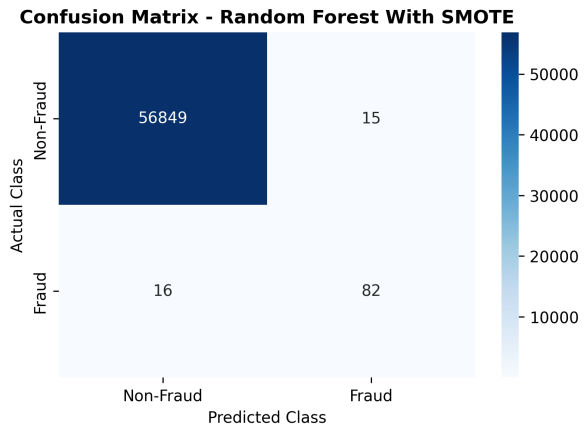

Figure 8: Confusion Matrix for Random Forest with SMOTE

Metric	Value
Accuracy	99.95%
Precision	84.54%
Recall	83.67%
F1 Score	84.10%
False Positives	15
False Negatives	16

Table 6: Random Forest with SMOTE Results

Random Forest with SMOTE slightly improved Recall from 81.63% to 83.67%. The number of false negatives decreased from 18 to 16, meaning the model detected two additional fraud cases.

However, this small Recall improvement came at the cost of reduced Precision. Precision decreased from 94.12% to 84.54%, and false positives increased from 5 to 15.

This indicates that SMOTE made Random Forest slightly more sensitive to fraud, but the improvement was not strong enough to outperform Random Forest without SMOTE. The model already handled the imbalanced dataset well, so the synthetic fraud samples did not provide substantial additional benefit.

5.5 Overall Model Comparison

Table 7 summarizes the results across all four experimental scenarios.

Model	Acc.	Prec.	Recall	F1
LR Without SMOTE	99.91	82.89	64.29	72.41
LR With SMOTE	97.42	5.80	91.84	10.92
RF Without SMOTE	99.96	94.12	81.63	87.43
RF With SMOTE	99.95	84.54	83.67	84.10

Table 7: Overall Model Performance Comparison

Figure 9 visually compares the major model evaluation metrics.

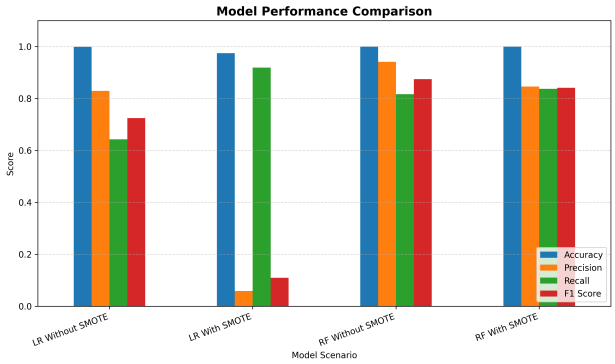

Figure 9: Comparison of Accuracy, Precision, Recall, and F1 Score

The comparison shows that Logistic Regression with SMOTE achieved the highest Recall, while Random Forest without SMOTE achieved the best overall balance across Precision, Recall, and F1 Score.

5.6 False Positive and False Negative Analysis

In fraud detection, false positives and false negatives have direct business implications.

- False Positives represent genuine transactions incorrectly flagged as fraud.
- False Negatives represent fraud transactions incorrectly classified as genuine.

Figure 10 shows the false positive comparison across models.

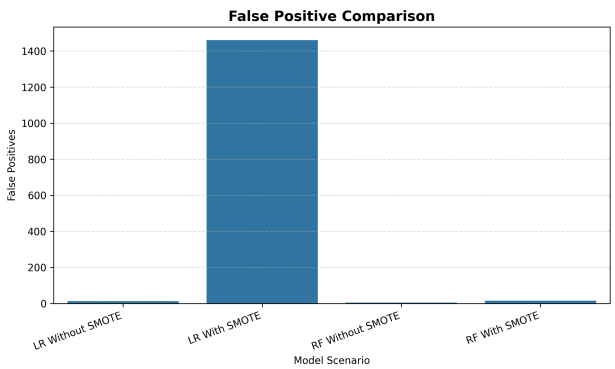

Figure 10: False Positive Comparison Across Models

Figure 11 shows the false negative comparison across models.

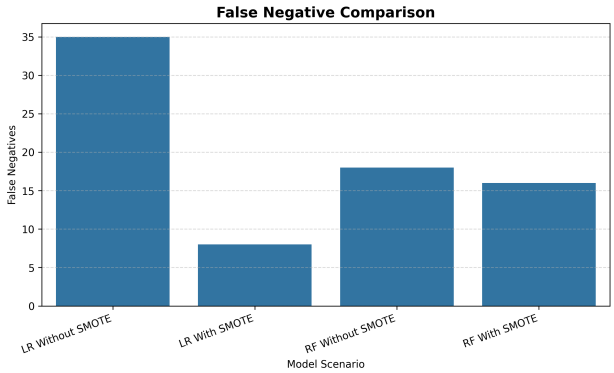

Figure 11: False Negative Comparison Across Models

Logistic Regression with SMOTE produced the lowest number of false negatives, reducing missed fraud cases to 8. However, it also produced the highest number of false positives, with 1461 genuine transactions incorrectly flagged as fraud.

Random Forest without SMOTE produced only 5 false positives and 18 false negatives. This makes it the most balanced and operationally practical model among the tested configurations.

5.7 Precision-Recall Tradeoff

The Precision-Recall tradeoff is central to fraud detection. A model that aggressively detects fraud may achieve high Recall but may also produce many false positives. On the other hand, a model with high Precision may generate fewer false alarms but may miss some fraud cases.

Figure 12 shows the Precision and Recall behavior of all four scenarios.

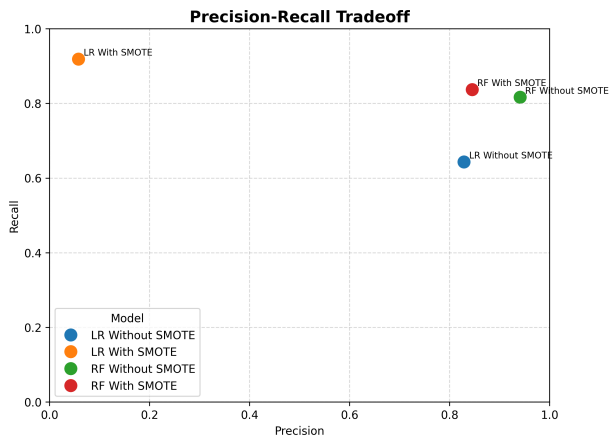

Figure 12: Precision-Recall Tradeoff Across Models

The plot clearly shows that Logistic Regression with SMOTE moved strongly toward high Recall but very low Precision. Random Forest without SMOTE maintained a better balance between the two metrics.

5.8 Why SMOTE Helped Logistic Regression

SMOTE helped Logistic Regression because Logistic Regression is a linear model and is highly influenced by the distribution of training samples.

Before SMOTE, the model was trained on a dataset dominated by non-fraud transactions. Since fraud samples were extremely rare, the learned decision boundary was biased toward the majority class. This caused many fraud cases to be classified as non-fraud.

After SMOTE, the training data became balanced. The model received many more synthetic fraud samples, which shifted the decision boundary toward better fraud detection. This improved Recall significantly.

However, the same shift also caused the model to classify many genuine transactions as fraud, leading to a large increase in false positives.

Therefore, SMOTE improved minority class sensitivity for Logistic Regression but reduced its practical precision.

5.9 Why SMOTE Did Not Improve Random Forest Significantly

Random Forest behaved differently because it is a nonlinear ensemble model.

Unlike Logistic Regression, Random Forest does not depend on a single linear decision boundary. It builds multiple decision trees, each learning different feature interactions and localized decision rules.

Because of this, Random Forest was already able to capture fraud patterns reasonably well even without SMOTE. Applying SMOTE provided only a small Recall improvement but introduced additional false positives.

This suggests that Random Forest was less dependent on artificial class balancing because its ensemble structure already handled the minority class more effectively than Logistic Regression.

5.10 Key Findings

The major findings from the experiments are:

- Logistic Regression without SMOTE achieved high Accuracy but moderate Recall.
- Logistic Regression with SMOTE achieved the highest Recall but produced too many false positives.

- Random Forest without SMOTE achieved the best overall balance between Precision, Recall, and F1 Score.
- Random Forest with SMOTE improved Recall slightly but reduced Precision and F1 Score.
- SMOTE was highly beneficial for Logistic Regression in terms of Recall, but not equally beneficial for Random Forest.

The strongest balanced model was Random Forest without SMOTE. However, if the only business objective is to catch the maximum number of fraud transactions, Logistic Regression with SMOTE achieved the lowest number of false negatives.

Overall, the results demonstrate that SMOTE is not universally beneficial for all algorithms. Its effect depends strongly on the underlying model architecture and decision boundary behavior.

6 Conclusion

This project presented a comparative analysis of the impact of Synthetic Minority Oversampling Technique (SMOTE) on Logistic Regression and Random Forest for fraud detection using highly imbalanced financial transaction data.

The experiments were conducted on a real-world credit card transaction dataset containing 284,807 transactions, where fraudulent transactions represented only approximately 0.17% of the total observations. Such severe class imbalance creates major challenges for Machine Learning models because the algorithms become strongly biased toward the majority non-fraud class.

Four experimental scenarios were evaluated:

1. Logistic Regression without SMOTE
2. Logistic Regression with SMOTE
3. Random Forest without SMOTE
4. Random Forest with SMOTE

The experimental results demonstrated that SMOTE had a significantly different impact on the two Machine Learning algorithms.

Logistic Regression without SMOTE achieved high Accuracy and strong Precision but relatively lower Recall. The model correctly identified most non-fraud transactions but missed a considerable number of fraud cases. This behavior occurred because Logistic Regression is a linear model and is highly influenced by class distribution. Since the dataset was dominated by non-fraud transactions, the learned decision boundary became biased toward the majority class.

After applying SMOTE, Logistic Regression showed a major improvement in Recall, increasing from 64.29% to 91.84%. This means the model became much more effective at detecting fraud transactions. The number of false negatives reduced significantly, indicating fewer fraud cases escaped detection.

However, this improvement came with a severe trade-off. Precision dropped dramatically because the model began classifying a large number of legitimate transactions as fraud. The number of false positives increased from 13 to 1461. This happened because SMOTE generated many synthetic fraud-like samples, causing Logistic Regression to shift its decision boundary aggressively toward the fraud class.

Random Forest behaved differently from Logistic Regression.

Even without SMOTE, Random Forest achieved strong fraud detection performance with high Precision, strong Recall, and the best overall F1 Score among all evaluated models. The model maintained low false positives while still detecting most fraud transactions effectively.

This stronger performance can be explained by the internal structure of Random Forest. Unlike Logistic Regression, Random Forest is a nonlinear ensemble model that builds multiple decision trees and captures complex feature interactions automatically. Because of this, Random Forest was already capable of learning minority fraud patterns reasonably well even from the imbalanced dataset.

After applying SMOTE, Random Forest showed only a small Recall improvement while Precision decreased slightly. Although the number of detected fraud transactions increased marginally, false positives also increased. This indicates that Random Forest was already sufficiently robust to handle the imbalance problem without requiring extensive synthetic oversampling.

The overall findings of this project demonstrate that:

- SMOTE significantly improved fraud sensitivity for Logistic Regression.
- SMOTE also caused Logistic Regression to become overly sensitive, generating a large number of false positives.
- Random Forest handled the imbalanced dataset effectively even without SMOTE.
- Applying SMOTE to Random Forest provided only marginal improvement while slightly reducing Precision.
- The effectiveness of SMOTE strongly depends on the underlying Machine Learning algorithm.

The project also highlighted the importance of evaluating fraud detection systems using multiple metrics rather than relying only on Accuracy. In highly imbalanced datasets, Accuracy alone can be misleading because models may achieve extremely high Accuracy while still failing to detect fraud effectively.

Among all evaluated configurations, Random Forest without SMOTE produced the most balanced and operationally reliable performance. However, Logistic Regression with SMOTE achieved the highest fraud detection sensitivity by minimizing false negatives.

Overall, this study demonstrates that imbalance handling techniques such as SMOTE should not be applied blindly. Their effectiveness depends heavily on the model architecture, learning behavior, and the trade-off between fraud detection capability and false alarm generation.

References

[1] Credit card fraud detection using machine learning algorithms. *Procedia Computer Science*, 2020.

[2] Random forest algorithm overview. 2024.

[3] Advanced fraud detection using machine learning and imbalanced data techniques. *Scientific Reports*, 2025.

[4] Per Harald Borgen. Logistic regression implementation repository, 2024.

[5] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[8] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *arXiv preprint arXiv:1106.1813*, 2011.

[9] GitHub Community. Awesome random forest resources, 2024.

[10] Analytical Minds Ltd. Smote variants library, 2024.

[11] Chao-Ying Joanne Peng, Kuk Lida Lee, and Gary M. Ingersoll. An introduction to logistic regression analysis and reporting. *The Journal of Educational Research*, 2002.